

Faculty of Mathematical Sciences
Ferdowsi University of Mashhad

Automata Theory Simulator

Design and Implementation of a DFA/NFA Simulation
and Conversion Framework

Instructor: Dr. Tabasi

Team Members : Fatemeh Abshang - Parsa Daroudi

Spring 1404

Submitted as partial fulfillment of the Foundations of Computation Theory course
requirements.

Contents

Abstract	4
1 Theoretical Background	4
1.1 Deterministic Finite Automaton	4
1.1.1 Extended Transition Function	5
1.1.2 Totality of the Transition Function	5
1.2 Nondeterministic Finite Automaton	5
1.2.1 ε -Closure	6
1.2.2 NFA Acceptance	6
1.3 Equivalence of DFA and NFA	6
1.4 DFA Minimization	7
2 Phase 1 — DFA Simulator	7
2.1 Problem Statement	7
2.2 Simulation Algorithm	7
2.3 Input Specification	7
2.4 Worked Example: Strings Ending with <i>ab</i>	8
3 Phase 2 — NFA to DFA Conversion	8
3.1 The Subset Construction Algorithm	8
3.2 Algorithm	8
3.2.1 Dead State	9
3.2.2 Complexity	9
3.3 Worked Example	9
4 DFA Minimization — Hopcroft's Algorithm	10
4.1 Motivation	10
4.2 Partition Refinement	10
4.2.1 Hopcroft's Key Optimization	10
4.3 Worked Example	10
5 System Architecture and Implementation	11
5.1 Project Structure	11
5.2 Module Responsibilities	12
5.3 Abstract Base Class	12
5.4 DFA Simulation	12
5.5 NFA: ε -Closure and Simulation	13
5.6 Subset Construction	13
5.7 Hopcroft's Algorithm	14
5.8 Input Validation	14
6 Command-Line Interface	15
6.1 Usage	15
6.2 Example Output	15
7 Web Interface	15
7.1 Architecture	15

7.2	API Endpoints	16
7.3	User Interface	16
7.4	Running the Application	16
8	Example Automata	16
8.1	DFA: Strings Ending with ab	17
8.2	DFA: Even Number of a 's	17
8.3	DFA: Contains 00	17
8.4	NFA: Starts with a OR Ends with b	17
8.5	NFA with ε -Transitions	17
9	Testing and Verification	17
9.1	Validation Tests	17
9.2	Simulation Correctness	18
9.3	NFA–DFA Equivalence	18
10	Summary and Conclusion	18
10.1	Key Insights	18
10.2	Possible Extensions	19

Abstract

This report presents the complete design and implementation of an interactive simulator for finite automata, covering both Deterministic Finite Automata (DFA) and Nondeterministic Finite Automata (NFA). The project is structured around two mandatory phases and two optional extensions, all built on a shared Python core that enforces correctness at every layer.

Phase 1 implements a DFA simulator. The user provides a complete machine definition — the set of states Q , the alphabet Σ , the transition function δ , the start state q_0 , and the set of accept states F — together with a test string w . The simulator reads w left to right, applies δ at each step, records the full execution trace $q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} q_n$, and outputs an **Accept** or **Reject** verdict depending on whether the final state belongs to F .

Phase 2 implements an NFA-to-DFA converter based on the *Subset Construction* (Power-set Construction) algorithm. The converter correctly handles both ε -transitions and purely nondeterministic transitions, and produces the complete specification of the equivalent DFA together with a mapping from each new DFA state to the NFA subset it represents.

Beyond the two required phases, the project adds three further contributions:

- **DFA Minimization** via Hopcroft’s partition-refinement algorithm. Given any DFA, the minimizer removes unreachable states and merges all pairs of indistinguishable states, producing the provably unique minimal DFA for the given regular language in $O(n \log n \cdot |\Sigma|)$ time.
- **Web Interface** with four interactive tabs — one per major operation. The interface is built with Flask on the backend and plain HTML/CSS/JavaScript on the frontend; it requires no external build tools and can be launched with a single command.
- **Input Validation Layer** that checks every JSON input for structural correctness before any automaton object is constructed, providing precise, human-readable error messages for every possible violation rather than cryptic Python tracebacks.

The entire implementation is written in Python 3, organized into eight single-responsibility modules with full type annotations. Five annotated example automata cover a range of DFA and NFA patterns and serve as both test cases and ready-to-run demonstrations.

1 Theoretical Background

1.1 Deterministic Finite Automaton

Definition 1.1 (DFA). A Deterministic Finite Automaton is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where:

- Q is a finite, non-empty set of states,
- Σ is a finite, non-empty alphabet,
- $\delta : Q \times \Sigma \rightarrow Q$ is the (total) transition function,
- $q_0 \in Q$ is the start state,
- $F \subseteq Q$ is the set of accept states.

The critical property of a DFA is that δ is *total* and *single-valued*: for every pair $(q, a) \in Q \times \Sigma$ there is exactly one next state $\delta(q, a)$. As a consequence, the DFA never “gets stuck” and never branches — it always follows a unique, deterministic path through its state graph.

Intuition: states as memory. The states of a DFA represent the machine’s *finite memory*: they encode exactly the information about the input seen so far that is needed to decide acceptance, and nothing more. For example, a DFA that accepts strings ending with ab needs to remember only three facts about the suffix of the input: (1) no progress toward ab , (2) the last character was a , or (3) the last two characters were ab . Three states suffice, regardless of how long the input is — this is the essence of why regular languages can be recognised with bounded memory.

1.1.1 Extended Transition Function

To define acceptance over strings we extend δ inductively to $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$:

$$\hat{\delta}(q, \varepsilon) = q \qquad \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$$

for all $q \in Q$, $a \in \Sigma$, $w \in \Sigma^*$. A string w is *accepted* by M if and only if $\hat{\delta}(q_0, w) \in F$. The *language* recognised by M is $L(M) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}$.

1.1.2 Totality of the Transition Function

Because δ must be defined for every (q, a) pair, a DFA that would otherwise have “missing” transitions is completed by adding an implicit *dead state* (also called a *sink* or *trap* state) $q_d \notin Q$ with $\delta(q_d, a) = q_d$ for all $a \in \Sigma$ and $q_d \notin F$. The dead state absorbs all further input without any possibility of acceptance.

Our implementation enforces totality *explicitly* at construction time: the DFA class checks that every (q, a) pair has a defined target, and raises a `ValueError` with a precise message if any entry is missing. This catches errors early, before any simulation is attempted.

1.2 Nondeterministic Finite Automaton

Definition 1.2 (NFA). A Nondeterministic Finite Automaton is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$ where all components are as in a DFA except that the transition function has signature

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \longrightarrow \mathcal{P}(Q).$$

There are two key generalisations over the DFA:

1. **Nondeterminism.** $\delta(q, a)$ may be any subset of Q : the empty set (this computation branch dies), a singleton (identical to the deterministic case), or a set of two or more states (the machine *branches* into parallel computations, each continuing independently).
2. **ε -transitions.** $\delta(q, \varepsilon)$ allows the machine to change state without consuming any input symbol, enabling free “jumps” between states.

The standard mental model for NFA execution is that the machine runs all branches *simultaneously*: at any point in time it is in a *set* of active states rather than a single state. A string is accepted if *at least one* of the active states at the end of input is an accept state. This is precisely the set-based simulation we implement.

Why NFAs are useful. Even though NFAs and DFAs recognise the same languages (Section 1.3), NFAs are often exponentially more concise. For example, the language “strings over $\{a, b\}$ that contain aa or end with bb ” can be expressed as a 5-state NFA by simply combining two small machines with ε -transitions, whereas the equivalent DFA may require significantly more states. NFAs are the natural output of algorithms that convert regular expressions to automata (Thompson’s construction), making them a crucial intermediate representation in practice.

1.2.1 ε -Closure

Definition 1.3 (ε -closure). *The ε -closure of a set $S \subseteq Q$ is*

$$\varepsilon\text{-closure}(S) = \{q \in Q \mid q \text{ is reachable from some } s \in S \text{ by zero or more } \varepsilon\text{-transitions}\}.$$

Computing ε -closure is a simple graph reachability problem (BFS or iterative DFS) on the subgraph formed by ε -edges. It is used in two places: (1) initialising the set of active states before any input is read, and (2) expanding the active set after each symbol is consumed.

1.2.2 NFA Acceptance

Given input $w = a_1a_2 \cdots a_n$, define the sequence of active-state sets:

$$R_0 = \varepsilon\text{-closure}(\{q_0\}), \quad R_i = \varepsilon\text{-closure}\left(\bigcup_{q \in R_{i-1}} \delta(q, a_i)\right), \quad i = 1, \dots, n.$$

N *accepts* w if and only if $R_n \cap F \neq \emptyset$.

1.3 Equivalence of DFA and NFA

Despite the apparent extra power of nondeterminism, DFAs and NFAs recognise exactly the same class of languages — the *regular languages*.

Theorem 1.1 (Rabin–Scott, 1959). *For every NFA N there exists a DFA M such that $L(M) = L(N)$.*

The constructive proof is the Subset Construction algorithm (Section 3), which we implement in Phase 2. The converse (every DFA is trivially an NFA with singleton transition sets and no ε -transitions) is immediate. Together, these results establish:

$$\mathcal{L}(\text{DFA}) = \mathcal{L}(\text{NFA}) = \text{Regular Languages}.$$

This equivalence is conceptually important: nondeterminism does *not* add expressive power to finite automata. It is, however, an extremely useful notational tool — NFAs are often exponentially more concise than the equivalent DFA, and many language descriptions arise more naturally as NFAs. The price paid is that NFA simulation requires tracking a set of active states, whereas DFA simulation tracks only a single state.

1.4 DFA Minimization

Every regular language has a unique (up to state renaming) minimum-state DFA. Minimization matters in practice because the DFA produced by Subset Construction can be exponentially larger than necessary: an NFA with n states can yield a DFA with up to 2^n states, many of which may be redundant.

Definition 1.4 (Distinguishable states). *States $p, q \in Q$ are distinguishable if there exists a string $w \in \Sigma^*$ such that exactly one of $\hat{\delta}(p, w)$ and $\hat{\delta}(q, w)$ belongs to F . In other words, the two states can be told apart by feeding them the same input and observing different outcomes. States that are not distinguishable are called equivalent or indistinguishable.*

Merging all equivalent states yields the minimal DFA. The theoretical foundation is the **Myhill–Nerode theorem**: the states of the minimal DFA correspond exactly to the equivalence classes of the right-congruence

$$x \sim_L y \iff \forall z \in \Sigma^* : xz \in L \iff yz \in L,$$

and this minimal DFA is unique up to state renaming.

2 Phase 1 — DFA Simulator

2.1 Problem Statement

Given a DFA $M = (Q, \Sigma, \delta, q_0, F)$ and an input string $w \in \Sigma^*$, determine whether M accepts w and produce the complete execution trace $q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} q_n$.

2.2 Simulation Algorithm

The algorithm is a direct implementation of the extended transition function $\hat{\delta}$, reading the input one symbol at a time:

1. Set current state $q \leftarrow q_0$.
2. For each symbol a_i in w (left to right):
 - (a) If $a_i \notin \Sigma$, raise an error (undefined symbol).
 - (b) Record the transition step $(q, a_i, \delta(q, a_i))$.
 - (c) Update: $q \leftarrow \delta(q, a_i)$.
3. **Accept** if $q \in F$; **Reject** otherwise.

Complexity. Each step performs a single lookup in the transition table (a Python dictionary), which runs in $O(1)$ expected time. The overall simulation therefore runs in $\Theta(|w|)$ time. Space for the trace is also $\Theta(|w|)$.

Edge cases. The empty string ε is handled correctly: the loop body never executes, the machine stays at q_0 , and the result is **Accept** if and only if $q_0 \in F$. A symbol not in Σ is rejected immediately with a clear error message rather than silently producing a wrong result.

2.3 Input Specification

The DFA is specified as a JSON object:

Field	Type	Meaning
states	list of strings	Q
alphabet	list of strings	Σ
start	string	q_0
accept	list of strings	F
transitions	nested object	$\text{transitions}[q][a] = \delta(q, a)$

2.4 Worked Example: Strings Ending with ab

$$Q = \{q_0, q_1, q_2\}, \quad \Sigma = \{a, b\}, \quad q_0 = q_0, \quad F = \{q_2\}$$

State	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

Simulation of $w = abab$:

$$q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{a} q_1 \xrightarrow{b} q_2$$

Final state $q_2 \in F \Rightarrow$ **ACCEPT**.

Simulation of $w = aaa$:

$$q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{a} q_1$$

Final state $q_1 \notin F \Rightarrow$ **REJECT**.

The three states track exactly the necessary information: whether the last symbol was nothing special (q_0), was a (q_1), or was b preceded by a (q_2). No additional state is needed, so this DFA is already minimal.

3 Phase 2 — NFA to DFA Conversion

3.1 The Subset Construction Algorithm

The Subset Construction converts $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ into an equivalent DFA $M = (Q_M, \Sigma, \delta_M, s_0, F_M)$ whose states are *subsets* of Q_N .

Definition 3.1 (DFA start state). $s_0 = \varepsilon\text{-closure}(\{q_0\})$.

Definition 3.2 (DFA transition). For $S \subseteq Q_N$ and $a \in \Sigma$:

$$\delta_M(S, a) = \varepsilon\text{-closure}\left(\bigcup_{q \in S} \delta_N(q, a)\right).$$

Definition 3.3 (DFA accept states). $F_M = \{S \in Q_M \mid S \cap F_N \neq \emptyset\}$. A DFA state is accepting if and only if it contains at least one NFA accept state.

3.2 Algorithm

1. Compute s_0 and add it to the worklist $\mathcal{W}$.
2. While $\mathcal{W} \neq \emptyset$:

- (a) Pop S from $\mathcal{W}$.
- (b) For each $a \in \Sigma$: compute $T = \delta_M(S, a)$.
- (c) If T is new, add it to $\mathcal{W}$.
- (d) Record the transition $\delta_M(S, a) = T$.

3. Mark all DFA states S with $S \cap F_N \neq \emptyset$ as accepting.

3.2.1 Dead State

If $T = \emptyset$ is produced for some (S, a) pair — meaning no NFA state survives after reading symbol a from the states in S — then $\emptyset$ becomes the *dead state* in the DFA: a non-accepting state with self-loops on every symbol ($\delta_M(\emptyset, a) = \emptyset$ for all $a \in \Sigma$). Once the DFA enters the dead state it can never reach an accept state. Our implementation adds the dead state (labelled $\emptyset$) *only when it is actually reachable*, keeping the output DFA as compact as possible.

3.2.2 Complexity

An NFA with n states can in the worst case produce 2^n DFA states, one for each subset of Q_N . However, most subsets are typically unreachable from the start state, so the actual number of DFA states is often much smaller. The algorithm processes each reachable subset once and performs $|\Sigma|$ lookups per subset, giving a time complexity of $O(2^n \cdot |\Sigma|)$ in the worst case and $O(|Q_M| \cdot |\Sigma|)$ in practice, where $|Q_M|$ is the number of reachable subsets.

3.3 Worked Example

Consider the NFA over $\{a, b\}$ with ε -transitions that accepts strings whose number of a 's is even *or* whose number of b 's is even:

$$Q_N = \{q_0, q_1, q_2, q_3, q_4\}, \quad F_N = \{q_1, q_3\}$$

State	ε	a	b
q_0	$\{q_1, q_3\}$	$\emptyset$	$\emptyset$
q_1	$\emptyset$	$\{q_2\}$	$\{q_1\}$
q_2	$\emptyset$	$\{q_1\}$	$\{q_2\}$
q_3	$\emptyset$	$\{q_3\}$	$\{q_4\}$
q_4	$\emptyset$	$\{q_4\}$	$\{q_3\}$

Step 1. $s_0 = \varepsilon\text{-closure}(\{q_0\}) = \{q_0, q_1, q_3\}$. Since $\{q_1, q_3\} \subseteq F_N$, the start state is accepting.

Step 2.

$$\begin{aligned} \delta_M(\{q_0, q_1, q_3\}, a) &= \varepsilon\text{-closure}(\{q_2, q_3\}) = \{q_2, q_3\} \\ \delta_M(\{q_0, q_1, q_3\}, b) &= \varepsilon\text{-closure}(\{q_1, q_4\}) = \{q_1, q_4\} \end{aligned}$$

Continuing the algorithm yields 4 reachable DFA states — far fewer than the theoretical maximum of $2^5 = 32$. Only the string ab (and any string with an odd number of both a 's and b 's) is rejected.

4 DFA Minimization — Hopcroft's Algorithm

4.1 Motivation

The DFA produced by Subset Construction can be much larger than necessary. Consider a simple example: if two DFA states p and q have identical transition behaviour and the same membership in F , they are indistinguishable and can safely be merged into a single state without changing the accepted language. Hopcroft's algorithm systematically finds *all* such pairs and merges them, producing the unique minimal DFA for the given language.

Before running the main algorithm we also remove *unreachable states* — states that can never be visited starting from q_0 — since they cannot affect the language and must not appear in the minimal DFA.

4.2 Partition Refinement

The algorithm maintains a *partition* $\mathcal{P}$ of the reachable states into groups of mutually indistinguishable states and iteratively splits groups that contain distinguishable states.

1. **Remove unreachable states** via BFS from q_0 .
2. **Initialize:** $\mathcal{P} = \{F, Q \setminus F\}$ (empty groups are discarded).
3. Add both groups to a worklist $\mathcal{W}$.
4. While $\mathcal{W} \neq \emptyset$:
 - (a) Pop a *splitter* C from $\mathcal{W}$.
 - (b) For each $a \in \Sigma$, let $\text{Pre}(C, a) = \{q \in Q \mid \delta(q, a) \in C\}$.
 - (c) For each group $G \in \mathcal{P}$: if $G \cap \text{Pre}(C, a) \neq \emptyset$ and $G \setminus \text{Pre}(C, a) \neq \emptyset$, split G into $G_1 = G \cap \text{Pre}(C, a)$ and $G_2 = G \setminus \text{Pre}(C, a)$. Update $\mathcal{W}$ accordingly.
5. Each final group is one state of the minimal DFA; choose any representative (we use the lexicographically smallest state name).

4.2.1 Hopcroft's Key Optimization

When splitting G into G_1 and G_2 :

- If G was already in $\mathcal{W}$, replace it with both G_1 and G_2 .
- Otherwise, add only the *smaller* of G_1 and G_2 to $\mathcal{W}$.

The intuition is that the larger half “inherits” the role of G as a splitter (anything that was split by G will also be split by its larger piece), while the smaller half introduces genuinely new splitting information. This bounds the total number of (group, symbol) pairs processed to $O(n \log n)$, giving the algorithm $O(n \log n \cdot |\Sigma|)$ overall time — significantly better than the naive $O(n^2)$ table-filling approach.

4.3 Worked Example

Consider a 5-state DFA where q_3 and q_4 are redundant copies of q_0 and q_1 (both unreachable from the start in normal operation but included for illustration):

State	<i>a</i>	<i>b</i>
q_0 (start)	q_1	q_0
q_1	q_1	q_2
q_2 (accept)	q_1	q_0
q_3	q_4	q_3
q_4	q_4	q_2

Initial partition: $\{\{q_2\}, \{q_0, q_1, q_3, q_4\}\}$.

After refinement, $q_0 \equiv q_3$ and $q_1 \equiv q_4$ are identified. The minimal DFA has 3 states, equivalent to the DFA of Section 2.4.

5 System Architecture and Implementation

5.1 Project Structure

```

project/
|-- core/
|   |-- automata.py      # Abstract base class
|   |-- dfa.py           # DFA + SimulationResult
|   |-- nfa.py           # NFA + epsilon_closure
|   |-- converter.py      # Subset Construction (NFA -> DFA)
|   '-- minimizer.py     # Hopcroft's minimization
|-- iohandler/
|   |-- parser.py        # JSON -> automata objects
|   '-- validator.py     # Structural validation
|-- api/
|   '-- app.py           # Flask REST API
|-- ui/
|   '-- index.html       # Web interface
|-- examples/           # Five annotated JSON automata
'-- main.py             # Command-line interface

```

5.2 Module Responsibilities

Module	Responsibility
<code>automata.py</code>	Abstract base class; validates non-empty states/alphabet, valid start and accept states.
<code>dfa.py</code>	DFA simulation; validates totality of δ ; produces <code>SimulationResult</code> with full trace.
<code>nfa.py</code>	NFA simulation; implements ε -closure via iterative DFS; <code>move</code> for symbol transitions.
<code>converter.py</code>	Subset Construction; returns <code>ConversionResult</code> with DFA and subset $\rightarrow$ name mapping.
<code>minimizer.py</code>	Hopcroft's algorithm; removes unreachable states first; returns <code>MinimizationResult</code> with partition map.
<code>validator.py</code>	Validates raw JSON before object construction.
<code>parser.py</code>	Loads JSON, calls validator, builds typed objects.
<code>app.py</code>	REST API bridging UI to core logic.

5.3 Abstract Base Class

All automata share structural invariants enforced in `Automata`:

```

1 class Automata(ABC):
2     def __init__(self, states, alphabet, start, accept):
3         self.states = states
4         self.alphabet = alphabet
5         self.start = start
6         self.accept = accept
7         self._validate()
8
9     def _validate(self):
10        if not self.states:
11            raise ValueError("State set cannot be empty.")
12        if not self.alphabet:
13            raise ValueError("Alphabet cannot be empty.")
14        if self.start not in self.states:
15            raise ValueError(f"Start state '{self.start}' not in states.")
16        invalid = self.accept - self.states
17        if invalid:
18            raise ValueError(f"Accept states not in states: {invalid}")
19
20        @abstractmethod
21        def run(self, input_string: str) -> object:
22            pass

```

Listing 1: Abstract base class `automata.py`.

5.4 DFA Simulation

```

1 def _validate_transitions(self):
2     # Ensure delta is total: every (state, symbol) has a target
3     for state in self.states:
4         for symbol in self.alphabet:

```

```

5         target = self.transitions.get(state, {}).get(symbol)
6         if target is None:
7             raise ValueError(
8                 f"Missing transition: d({state}, {symbol}).")
9         if target not in self.states:
10            raise ValueError(
11                f"d({state}, {symbol}) -> '{target}' invalid.")
12
13 def run(self, input_string):
14     current, steps = self.start, []
15     for symbol in input_string:
16         if symbol not in self.alphabet:
17             raise ValueError(f"Symbol '{symbol}' not in alphabet.")
18         nxt = self.transitions[current][symbol]
19         steps.append(SimulationStep(current, symbol, nxt))
20         current = nxt
21     return SimulationResult(
22         accepted = current in self.accept,
23         final_state = current,
24         steps = steps,
25     )

```

Listing 2: Transition validation and simulation in dfa.py.

5.5 NFA: ϵ -Closure and Simulation

```

1 def epsilon_closure(self, states):
2     closure, stack = set(states), list(states)
3     while stack:
4         state = stack.pop()
5         for t in self.transitions.get(state, {}).get(EPSILON, set()):
6             if t not in closure:
7                 closure.add(t)
8                 stack.append(t)
9     return closure
10
11 def run(self, input_string):
12     current = self.epsilon_closure({self.start})
13     for symbol in input_string:
14         current = self.epsilon_closure(self.move(current, symbol))
15         if not current:
16             # dead -- no surviving path
17             return NFASimulationResult(
18                 accepted=False, final_states=set())
19     return NFASimulationResult(
20         accepted = bool(current & self.accept),
21         final_states = current,
22     )

```

Listing 3: epsilon-closure and simulation in nfa.py.

5.6 Subset Construction

```

1 def nfa_to_dfa(nfa):
2     start = frozenset(nfa.epsilon_closure({nfa.start}))
3     seen = {start: label(start)}
4     trans = {}
5     queue = [start]

```

```

6
7     while queue:
8         S = queue.pop()
9         sname = seen[S]
10        trans[sname] = {}
11        for a in sorted(nfa.alphabet):
12            T = frozenset(nfa.epsilon_closure(nfa.move(set(S), a)))
13            if T not in seen:
14                seen[T] = label(T)
15                queue.append(T)
16                trans[sname][a] = seen[T]
17
18        # Accept states contain at least one NFA accept state
19        dfa_accept = {name for sub, name in seen.items()
20                      if sub & nfa.accept}
21        ...

```

Listing 4: Core loop of Subset Construction in converter.py.

5.7 Hopcroft's Algorithm

```

1 def minimize(dfa):
2     reachable = _reachable_states(dfa)
3     partitions = [p for p in [frozenset(dfa.accept & reachable),
4                                frozenset(reachable - dfa.accept)] if p]
5     worklist = list(partitions)
6
7     while worklist:
8         C = worklist.pop()
9         for a in dfa.alphabet:
10            pre = frozenset(s for s in reachable
11                           if dfa.transitions[s][a] in C)
12            next_p = []
13            for G in partitions:
14                inside, outside = G & pre, G - pre
15                if inside and outside:
16                    next_p.extend([inside, outside])
17                    if G in worklist:
18                        worklist.remove(G)
19                        worklist.extend([inside, outside])
20                else:
21                    # Add smaller half (Hopcroft's optimization)
22                    worklist.append(
23                        inside if len(inside) <= len(outside)
24                        else outside)
25            else:
26                next_p.append(G)
27            partitions = next_p
28        ...

```

Listing 5: Partition refinement in minimizer.py.

5.8 Input Validation

The validator module checks the raw JSON before any object is constructed:

Check	Description
Required fields	<code>states</code> , <code>alphabet</code> , <code>start</code> , <code>accept</code> , <code>transitions</code> must all be present.
Non-empty lists	<code>states</code> and <code>alphabet</code> must be non-empty.
String elements	All items in <code>states</code> and <code>alphabet</code> must be strings.
DFA target type	<code>transitions[q][a]</code> must be a single string.
NFA target type	<code>transitions[q][a]</code> must be a list of strings.

Any violation raises a `ValueError` with a precise, human-readable message rather than a cryptic Python traceback.

6 Command-Line Interface

In addition to the web interface, the project provides a command-line interface through `main.py` that supports all four operations directly from the terminal. This is useful for scripting, batch testing, and running the simulator without starting a web server.

6.1 Usage

```
python main.py dfa      <json_path> <input_string>
python main.py nfa      <json_path> <input_string>
python main.py convert  <nfa_json_path>
python main.py minimize <dfa_json_path>
```

6.2 Example Output

Running the DFA simulator on the `dfa_ends_with_ab.json` example:

```
$ python main.py dfa examples/dfa_ends_with_ab.json abab
```

```
Input   : 'abab'
Result  : ACCEPT
Final   : q2
Trace   :
    delta(q0, a) -> q1
    delta(q1, b) -> q2
    delta(q2, a) -> q1
    delta(q1, b) -> q2
```

All errors (missing file, invalid JSON, missing transition, unknown symbol) are caught and reported with a clear `Error:` prefix, and the program exits with code 1, making it suitable for use in shell scripts.

7 Web Interface

7.1 Architecture

The web interface follows a two-tier architecture:

- **Backend** — Flask REST API (`api/app.py`), four endpoints, delegates all computation to the `core` modules.
- **Frontend** — Single HTML file (`ui/index.html`), no build step required, pure HTML/CSS/JavaScript.

7.2 API Endpoints

Method	Endpoint	Description
POST	<code>/api/dfa/simulate</code>	Simulate DFA; return trace and verdict.
POST	<code>/api/nfa/simulate</code>	Simulate NFA; return final states and verdict.
POST	<code>/api/nfa/convert</code>	Convert NFA to DFA; return DFA spec and subset map.
POST	<code>/api/dfa/minimize</code>	Minimize DFA; return minimal DFA and partition map.

7.3 User Interface

The UI is organized into four tabs:

1. **DFA Simulator** — Enter states, alphabet, start/accept states, and test string. “Build Transition Table” generates an editable $|\Sigma|$ -column table. “Run Simulation” calls the API and renders the step-by-step trace with colour-coded states and symbols.
2. **NFA Simulator** — Same workflow as DFA, but each cell accepts a comma-separated list of target states and an extra ε column appears automatically.
3. **NFA $\rightarrow$ DFA Converter** — Enter an NFA and click Convert. The result panel shows each DFA state with its originating NFA subset, which states are accepting, and the complete transition table.
4. **DFA Minimizer** — Enter a DFA and click Minimize. The result panel shows the number of states before and after, the partition map listing which original states merged, and the minimized transition table.

Each tab includes a **Load Example** button that pre-fills the form with a non-trivial automaton and builds the transition table automatically, allowing immediate experimentation without manual data entry.

7.4 Running the Application

```
pip install flask
python api/app.py
# Open http://localhost:5000 in a browser
```

8 Example Automata

File	Type	Alphabet	Language
dfa_ends_with_ab.json	DFA	$\{a, b\}$	$\{w \mid w \text{ ends with } ab\}$
dfa_even_a.json	DFA	$\{a, b\}$	$\{w \mid w _a \equiv 0 \pmod{2}\}$
dfa_contains_00.json	DFA	$\{0, 1\}$	$\{w \mid 00 \text{ is a substring of } w\}$
nfa_starts_a_or_ends_b.json	NFA	$\{a, b\}$	$\{w \mid w[0] = a\} \cup \{w \mid w[-1] = b\}$
nfa_epsilon_even_a_or_even_b.json	NFA+ ε	$\{a, b\}$	$\{w \mid w _a \text{ even}\} \cup \{w \mid w _b \text{ even}\}$

8.1 DFA: Strings Ending with ab

Three states track the minimal necessary suffix: q_0 (no progress), q_1 (last char was a), q_2 (last two chars were ab). This DFA is already minimal — all three states are distinguishable by the strings ε , b , and ab respectively.

8.2 DFA: Even Number of a 's

A two-state toggle: q_0 (even count of a 's seen, **accept**) and q_1 (odd count). Reading b does not change the parity; reading a flips it. The empty string is accepted because zero is even.

8.3 DFA: Contains 00

Three states track progress toward finding 00: q_0 (no leading 0 seen), q_1 (one 0 seen, waiting for another), q_2 (00 found — accept and stay here forever). Once in q_2 the machine ignores all further input.

8.4 NFA: Starts with a OR Ends with b

From q_0 , reading a branches into two paths simultaneously: one tracking “started with a ” and one tracking future suffixes. This illustrates how NFA branching cleanly expresses the union of two languages without the explicit case analysis required in a DFA.

8.5 NFA with ε -Transitions

From q_0 , two ε -transitions split the machine into two independent sub-DFAs: one counting a 's modulo 2, one counting b 's modulo 2. The result is accepted if either counter is in its even (accept) state. The only rejected strings are those where *both* $|w|_a$ and $|w|_b$ are odd, e.g. ab , ba , $abba$.

9 Testing and Verification

9.1 Validation Tests

Each module was tested for all error conditions:

- Empty state set or alphabet.
- Start state not in Q .
- Accept state not in Q .
- Missing transition entry (DFA only).
- Undeclared state appearing in the transition table.

- DFA target is a list instead of a string (and vice versa for NFA).
- Symbol in input string not in Σ .

9.2 Simulation Correctness

Automaton	String	Expected	Result
DFA ends- <i>ab</i>	abab	ACCEPT	✓
DFA ends- <i>ab</i>	aaa	REJECT	✓
DFA ends- <i>ab</i>		REJECT	✓
DFA even- <i>a</i>	aa	ACCEPT	✓
DFA even- <i>a</i>	a	REJECT	✓
DFA even- <i>a</i>		ACCEPT	✓
DFA contains-00	1001	ACCEPT	✓
DFA contains-00	01	REJECT	✓
NFA starts- <i>a</i> /ends- <i>b</i>	aba	ACCEPT	✓
NFA starts- <i>a</i> /ends- <i>b</i>	ba	REJECT	✓
NFA ε	aa	ACCEPT	✓
NFA ε	ab	REJECT	✓

9.3 NFA–DFA Equivalence

For each NFA example, after Subset Construction we verified programmatically that the NFA and the converted DFA produce identical verdicts on every test string. This confirms the correctness of the ε -closure computation and the transition mapping.

10 Summary and Conclusion

This project demonstrates the full pipeline of finite automata theory through a working, well-tested, and interactive implementation. Table 1 summarises all components.

Component	Phase	Time Complexity	Status
DFA Simulation	1 (mandatory)	$\Theta(w)$	Complete
NFA Simulation	1 (mandatory)	$\Theta(w \cdot Q)$	Complete
NFA-to-DFA Conversion	2 (mandatory)	$O(2^{ Q } \cdot \Sigma)$	Complete
DFA Minimization	extension	$O(n \log n \cdot \Sigma)$	Complete
Web Interface	extension	—	Complete
Input Validation	—	$O(Q \cdot \Sigma)$	Complete

Table 1: Summary of all implemented components.

10.1 Key Insights

States as memory. The most important idea in DFA design is that states represent exactly the information about past input that is relevant to future decisions. Identifying the right state set — not too large, not too small — is the central design challenge.

Nondeterminism as a design tool. NFAs are often the natural description of a language (“accept strings that contain 00 *or* end with *ab*”), but simulation requires tracking

a set of active states rather than a single state. The Subset Construction makes this set-tracking explicit in the DFA itself.

Minimality is unique. The Myhill–Nerode theorem guarantees that there is exactly one minimal DFA for any regular language (up to state renaming). Hopcroft’s algorithm finds it efficiently, and its output can be used as a canonical form for comparing or storing regular languages.

The three algorithms form a pipeline. Given a language description as an NFA (possibly with ε -transitions), one can:

1. Convert to DFA via Subset Construction (Phase 2),
2. Minimize the DFA via Hopcroft’s algorithm (extension),
3. Simulate any input string on the minimal DFA (Phase 1).

This pipeline — implemented in full in this project — is the standard foundation for lexer construction, regular expression matching, and formal verification of finite-state systems.

10.2 Possible Extensions

Several natural extensions could build on this framework:

- **Regular expression to NFA.** Implementing Thompson’s construction would allow the user to type a regular expression (e.g. $(a|b)^*ab$) and automatically produce an NFA, which can then be converted and minimized using the existing pipeline.
- **Automata equivalence checking.** Two DFAs M_1 and M_2 are equivalent if and only if their minimal DFAs are isomorphic. Adding a comparison feature would allow users to verify whether two different machine descriptions recognise the same language.
- **Visual state diagram.** Rendering the automaton as a directed graph (using a library such as Graphviz or D3.js) and animating the active state during simulation would make the tool significantly more educational.
- **Automata learning (L^* algorithm).** Angluin’s L^* algorithm can infer an unknown DFA from a series of membership and equivalence queries. Integrating this would turn the simulator into an interactive learning environment.